

Monster Lab

The Tuesday story: build Git, one problem at a time.

Each lab builds one monster. Every button runs real Git.

Feel the problem. Invent the fix. See what Git does behind the door.

STEP 0

Everyone, one monster

PROBLEM

Everyone edits one monster at once. "Who changed the legs?" Nobody knows.

IDEA

Save every version, with a name on it.

GIT

None yet. This is life without it.

Monster Lab • Lab 1 • Ana • Step 0 • Everyone, one monster

STEP 0

Everyone, one monster

Your lab shares one monster.
Change any part, any time. Go!

We'll ask: *What did your monster look like a minute ago? Who changed the legs?*

► Behind the door

what Git just did

Your lab's monster



FACE
frog



BODY
box



LEGS
wheels

STEP 1

Save every version

PROBLEM

You could not get an old monster back.

IDEA

Never erase. Every save is a new card: the whole monster, your name, the card before.

GIT

git commit. The card's ID is a hash of all that. Following the arrows back is git log.

Monster Lab • Lab 1 • Ana • Step 1 • Save every version

STEP 1

Save every version

The old monster is gone for good. From now on, every save makes a card. Change one part, then press **Save card**. Take turns. Everyone saves at least once.

✓ Everyone saved a card (3/3)

Bonus • Click the oldest card. Who made it? What is its parent?

We'll ask: Why do arrows point back, never forward?

> Behind the door what Git just did

Your draft

You're on: main

 FACE
frog

 BODY
coat

 LEGS
duck

Save card
git commit

NEW

All saved

Your lab's cards git log

235b510
Monster Lab
Start

← a3dcd73
Ana
FACE: smiley →
frog

← f428826
Raj
BODY: box → coat

← 7b44fa2
Mei
LEGS: sticks →
duck

← points to the card before sticky note YOU where you are

STEP 2

Try two ideas at once

PROBLEM

Two risky ideas could wreck the good monster.

IDEA

Each pair gets its own sticky note. Saving moves only theirs.

GIT

git branch: a tiny file holding one card's ID. Your pin (HEAD) is the note you are on.

Monster Lab • Lab 1 • Ana • Step 2 • Try two ideas at once

STEP 2

Try two ideas at once

Try two ideas without losing main. Each pair makes its own sticky note. Build your idea on it and save.

YOUR MISSION

Press **New sticky note**, name it **cat-robot** (partner: **Switch to cat-robot**). FACE → 🐱, BODY → 🤖. **Save card**.
You're in Pair A ([change](#))

✔ cat-robot has 🐱 + 🤖

✔ superhero has 🦸 + 🦋

We'll ask: *Where is the original monster now? Did anything get copied?*

> Behind the door

what Git just did

Your draft

You're on: cat-robot

FACE

cat

BODY

robot

LEGS

duck

All saved

New sticky note

git switch -c

NEW

Switch to

superhero

git switch

NEW

Save card

git commit

Your lab's cards

git log

main

a3dcd73 Ana

FACE: smiley → frog

f428826 Raj

BODY: box → coat

7b44fa2 Mei

LEGS: sticks → duck

cat-robot YOU

efa00f8 Mei

FACE: frog → cat, BODY: coat...

superhero

a215e91 Raj

BODY: coat → superhero, LEGS...

← points to the card before

sticky note

YOU where you are

STEP 3

Make one monster from both

PROBLEM

The client wants one monster with both ideas.

IDEA

Compare each part with where both ideas started. Changed on one side: keep it.

GIT

git merge. FACE and LEGS combine alone. BODY changed on both sides: a person decides.

Monster Lab • Lab 1 • Ana • Step 3 • Make one monster from both

STEP 3

Make one monster from both

One person presses, everyone watches. Swap each step.

The client wants one monster with both ideas. On **main**: merge **cat-robot**. Then merge **superhero**.

- ✓ main has cat-robot
- main has superhero (merge card)

We'll ask: Why did the first merge only move the note? Why did BODY need you?

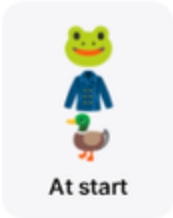
Behind the door
what Git just did

Merging superhero into main. BODY needs a choice.

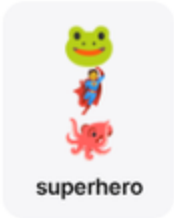
Open

Merging superhero into main

Both sides changed these parts since the start. Pick one for each.

 At start

 main

 superhero

FACE ✓ cat

BODY  robot (main)  superhero (superhero) [Pick another...](#)

LEGS ✓ tentacles

Finish merge Cancel merge `git merge --abort`

Behind the door • the file Git wrote

superhero

a215e91
Raj
BODY: coat →
superhero, LEGS...

STEP 4

Meet the Wall

PROBLEM

Each lab's cards live only in that lab.

IDEA

One shared copy for the class, like GitHub. Every lab gets a full copy.

GIT

git clone. The same card has the same ID on every laptop: the ID is computed from it.

STEP 4

Meet the Wall

👁️ One person presses, everyone watches. Swap each step.

Each lab's cards lived only in that lab. The Wall is the class's shared copy, like GitHub. Your lab's cards are now a full copy of it (Lab 2's monster). Compare your newest card's ID with the Wall's.

We'll ask: *Only one lab made that card. Why is the ID the same everywhere?*

- Behind the door
 - what Git just did

Your draft

You're on: **main**

 **FACE**
cat

 **BODY**
shell

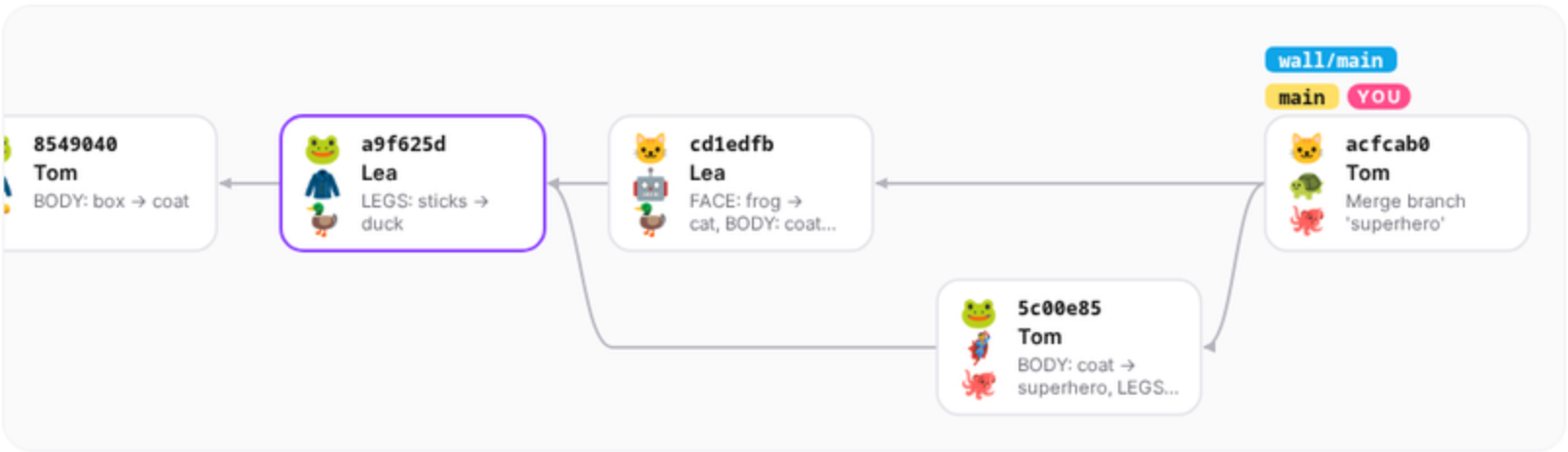
 **LEGS**
tentacles

Save card
git commit

New sticky note
git switch -c

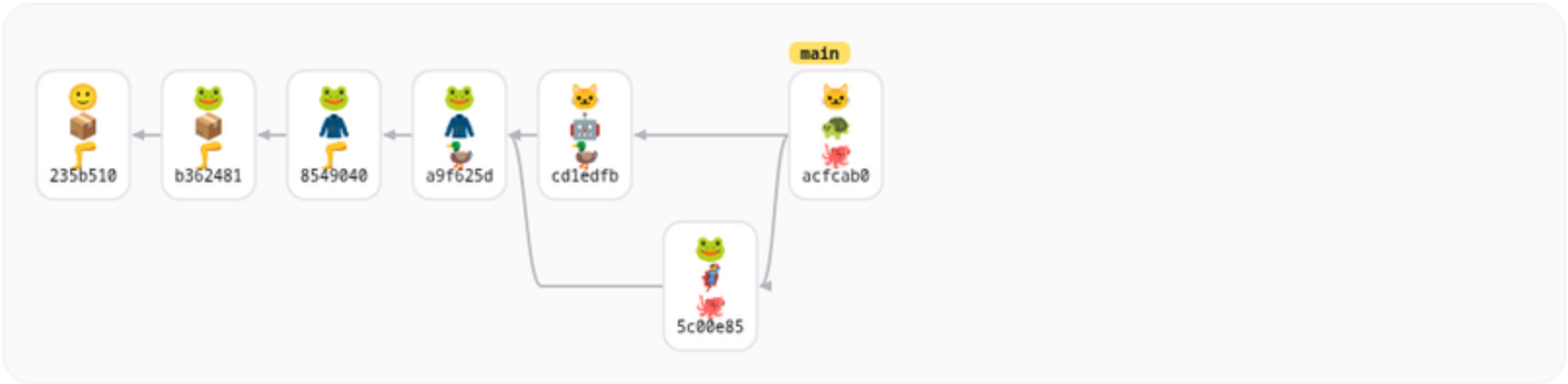
All saved

Your lab's cards git log



← points to the card before **main** sticky note **YOU** where you are **wall/main** the Wall, last time you checked

• The Wall the class's shared copy



STEP 5

Put your monster on the Wall

PROBLEM

The Wall refused your card.

IDEA

The Wall never drops a card. Get its new cards, combine, then send.

GIT

`git push` only moves the Wall forward. `git pull` = `git fetch` + `git merge`.

Monster Lab • Lab 2 • Priya • Step 5 • Put your monster on the Wall

Wall

☐ Your lab's cards match the Wall

We'll ask: *Why did the Wall refuse your card instead of adding it?*

Behind the door

what Git just did

PROBLEM

The Wall refused your card.

IDEA

The Wall never drops a card. Combine its new cards first.

GIT

`git push` only moves the Wall forward. `git pull` = `git fetch` + `git merge`.

You'd type: `git push` `git pull`

Last • Priya

Get & combine → merge card 27afdc3

`git fetch wall`
`git merge wall/main`

Git fetched the Wall's cards into wall/main, then merged. Each part changed on one side only. Git made merge card 27afdc3 with two parents.

Show the low-level steps Git ran

Your draft

You're on: **main**

FACE
dragon

BODY
shell

LEGS
skates

All saved

Send to Wall

NEW

`git push`

Get & combine

NEW

`git fetch`
`git merge`

Save card

`git commit`

New sticky note

`git switch -c`

Your lab's cards

git log

STEP 6

Oops: undo a shared mistake

PROBLEM

A mustache card reached the Wall, and every lab has it.

IDEA

Don't rip out a shared card. Add a card that undoes it.

GIT

git revert adds a fix card. git reset moves your note back. git reflog remembers every move.

Monster Lab • Lab 1 • Ana • Step 6 • Oops: undo

STEP 6

Oops: undo a shared mistake

One person presses, everyone watches. Swap each step.

A mustache card reached the Wall. Every lab has a copy. Remove it without breaking anyone's copy.

YOUR MISSION

Press **Get & combine**. Click the mustache card, press **Undo this card**, then **Send to Wall**.

☒

 You got the mustache card

☐

 Your lab's cards match the Wall, with no mustache

We'll ask: *Why is adding a fix card safe, but moving back is not?*

Behind the door

what Git just did



656e5df

656e5df8a5d45564f5acc1cc857ec01b78f4555f

Tiny style fix

Made by The Intern · just now

Parent 892658d

Show what Git stored

```
$ git cat-file -p 656e5df8a5d45564f5acc1cc857ec01b78f4555f
```

```
tree 739fc456d9ebb0e1f55ae0c500fba3da5e89da09
parent 892658d59e32f7637488352b55b91139fe976aff
author The Intern <intern@monster.lab> 1791149084 +0000
committer The Intern <intern@monster.lab> 1791149084 +0000

Tiny style fix
```

```
$ git cat-file -p 656e5df8a5d45564f5acc1cc857ec01b78f4555f:monster.txt
```

```
face: mustache
---
body: donut
---
legs: paws
```

Undo this card

git revert 656e5df

Move my note back here

git reset --hard 656e5df

main is already here.

NEW

note

Send to Wall

git push

wall/main

main YOU

892658d Sam

Merge remote-tracking...

656e5df The Intern

Tiny style fix

STEP 7

The boss wants it clean

PROBLEM

"Who added the tentacles?" Before the clean-up: Tom, Lab 2. After: not found.

IDEA

Rewriting history makes new cards and forgets the old ones.

GIT

Squash = one new card, new ID. git push --force makes the Wall forget. git gc deletes. The Tuesday paper.

Step 7 · The boss wants it clean

1:55

Watch the Wall. Who added the tentacles? Find out in your lab's cards.

THE WALL



BEFORE THE CLEAN-UP · WHO FIRST ADDED LEGS 🦵?

Wall: Tom (Lab 2), 5c00e85

Lab 1: Tom (Lab 2), 5c00e85 · Lab 2: Tom (Lab 2), 5c00e85 · Lab 3: Tom (Lab 2), 5c00e85

AFTER THE CLEAN-UP · WHO FIRST ADDED LEGS 🦵?

Wall: not found

Lab 1: not found · Lab 2: Tom (Lab 2), 5c00e85 · Lab 3: Tom (Lab 2), 5c00e85

Lab 1



✓ The Wall has one clean card

Force push

Lab 2



✓ The Wall has one clean card

Force push

Lab 3



✓ The Wall has one clean card

Force push

STEP 8

What you built

Cards never change.

Sticky notes move.

The Wall copies cards.
That's Git.

What you built

Cards never change. Sticky notes move. The Wall
copies cards. That's Git.

Lab	Cards saved	Merges	Conflicts solved	Pushes	Refused pushes	Reverts	Git ideas used
● Lab 1	7	5	1	3	0	1	Save 7 Branch 2 Fast-forward 4 Merge Conflict solved Push 3 Refused push Pull 5 Revert Reset Diary Force push
● Lab 2	6	9	1	1	4	0	Save 6 Branch 2 Fast-forward 7 Merge 2 Conflict solved Push Refused push 4 Pull 7 Revert Reset Diary Force push
● Lab 3	7	4	1	2	4	0	Save 7 Branch 2 Fast-forward Merge 3 Conflict solved Push 2 Refused push 4 Pull 2 Revert Reset Diary Force push

TEACHERS

What you see while it runs

SCRIPT

Each step's problem, the one question, and the answer you hope to hear.

LIVE

Every lab's cards, goals and alerts like "Refused twice in a row".

FEED

Who did what, with the real Git command.

Monster Lab · Teacher

Step 5 · Put your monster on the Wall

0:11 / 5:00 · 42 min ahead

Restart timer

WATCH

Watch for: the first lab gets in. The others are refused, then combine with no red. "Refused twice"? Check they pressed Get & combine.

PROBLEM

The Wall refused your card.

PAUSE · ASK

Why did the Wall refuse your card instead of adding it?

Ask on the projector

HOPE TO HEAR

It would drop the first lab's card. The Wall only moves forward.

IDEA

The Wall never drops a card. Combine its new cards first.

HOW GIT DOES IT

git push only moves the Wall forward. git pull = git fetch + git merge.

NEXT · STEP 6 · OOPS: UNDO A SHARED MISTAKE

PROBLEM

A bad card is on the Wall, and every lab has it.

ASK FIRST

After Sabotage and Get & combine: "How do you get rid of it?"

IDEA

Don't rip out a shared card. Add a card that undoes it.

Break · 4 min

Prev

Start Step 6

Adds Undo, Move back and the Safety diary. Press Sabotage first, on Step 5. If you didn't, Next does.

http://127.0.0.1:3102/
9 people · 9 online
Labs 3

- Sabotage
- Audit: who added
- Empty the Wall's bin
- git gc
- Reset session
- Open projector

Step 5 · Put your monster on the Wall

Lab 1

3 people

✓ Your lab's change is on the Wall
✓ Your lab's cards match the Wall

Ana A main
Raj B main
Mei A main

Move
Move
Move

Ana · Get & combine → fast-forward · 0:05 ago

Save 7 · Branch 2 · Fast-forward 3 · Merge
Conflict solved · Push 2 · Refused push · Pull 3

Lab 2

3 people

✓ Your lab's change is on the Wall
✓ Your lab's cards match the Wall

Priya A main
Tom B main
Lea A main

Move
Move
Move

Priya · Get & combine → fast-forward · 0:05 ago

Save 6 · Branch 2 · Fast-forward 4 · Merge 2
Conflict solved · Push · Refused push 3 · Pull 4

Lab 3

3 people

✓ Your lab's change is on the Wall
✓ Your lab's cards match the Wall

Sam A main
Kim B main
Ola A main

Move
Move
Move

Sam · Send to Wall → sent 892658d · 0:05 ago

Save 7 · Branch 2 · Fast-forward · Merge 3
Conflict solved · Push 2 · Refused push 4 · Pull 2

The Wall wall.git

Feed

- 9:24 PM Lab 2 · Priya — Get & combine → fast-forward git fetch wall git merge wall/main
- 9:24 PM Lab 1 · Ana — Get & combine → fast-forward git fetch wall git merge wall/main
- 9:24 PM Lab 3 · Sam — Send to Wall → sent 892658d git push
- 9:24 PM Lab 3 · Sam — Get & combine → merge card 892658d git fetch wall git merge wall/main
- 9:24 PM Lab 2 · Priya — Get & combine → fast-forward git fetch wall git merge wall/main
- 9:24 PM Lab 1 · Ana — Get & combine → nothing new git fetch wall git merge wall/main
- 9:24 PM Lab 3 · Kim — Send to Wall → refused (fetch first) git push
- 9:24 PM Lab 2 · Tom — Send to Wall → refused (fetch first) git push
- 9:24 PM Lab 3 · Ola — Send to Wall → refused (fetch first) git push
- 9:24 PM Lab 2 · Priya — Send to Wall → refused (fetch first) git push
- 9:24 PM Lab 3 · Sam — Send to Wall → refused (fetch first) git push
- 9:24 PM Lab 1 · Ana — Send to Wall → sent 97451dd git push
- 9:24 PM Lab 3 · Kim — Save card → card 7b2fa36 git commit -am 'LEGS: skates - paws'